Pre-Course Setup Guide

This guide is for participants to complete before the LSE PhD workshop. You only need this document, a laptop, and about 60-90 minutes tops. It is a pre-requisite for the workshop: as we only have 4 hours, we cannot be spending time on the setup.

The goal is simple: arrive with one working agentic coding tool, Python, Node.js, Git, and Playwright. The course repository and exercises will be shared later.

Prices below are public USD prices checked on 15 May 2026. They can change, and some providers localize prices or add VAT at checkout. Verify the current price on the official pricing page before subscribing.

If you have trouble with any step, contact me through the MS Teams chat dedicated to this workshop. You should already have access to that chat.

1. Pick Your Poison

You should choose one main agentic coding tool for the workshop. Do not install everything just because everything exists. Installing one tool well is better than arriving with three half-configured tools.

The workshop supports:

- Codex
- Claude Code
- Cursor

All three can read and edit code, run terminal commands, help debug errors, and use project instructions. The differences are mostly workflow and pricing.

Quick Recommendation

If this sounds like you	Choose
You want to follow the instructor demos as closely as possible	Codex
You already use ChatGPT heavily for research, writing, or data work	Codex
You want a strong terminal agent and good long-form reasoning	Claude Code
You expect to review papers, write prose, and refactor scripts in the same workflow	Claude Code
You want an IDE-first experience and prefer not to live in the terminal	Cursor
You use VS Code-style editors and want autocomplete plus agents	Cursor

If this sounds like you	Choose
You will mainly do light course exercises	Codex Plus, Claude Pro, or Cursor Pro
You will use agents daily on research code	Codex Pro $100, Claude Max 5x, or Cursor Pro+
You will run parallel agents on large codebases or heavy automation	Codex Pro $200, Claude Max 20x, or Cursor Ultra

For most PhD students in Economics, the pragmatic choices are:

- Cursor Pro if you want the smoothest editor experience.
- Claude Pro if you want the cheapest serious Claude Code entry point.
- ChatGPT Plus with Codex if you already use ChatGPT and want the course demos to match closely.

Upgrade only if you actually hit limits during real work. A 4-hour workshop does not require a premium $100 or $200 plan.

Codex Pricing

Official pages:

- Pricing: https://chatgpt.com/pricing/
- Pro tiers: https://help.openai.com/en/articles/9793128-about-chatgpt-pro-plans
- Codex CLI: https://developers.openai.com/codex/cli
- Codex desktop startup guide: https://openai.com/academy/codex-how-to-start/

Tier	Price	What it means for this workshop
ChatGPT Free	$0/month	Limited Codex access. Fine for reading this guide, not reliable for the hands-on work.
ChatGPT Go, where available	Regional price	Lower-cost plan in selected markets. Do not rely on it for the workshop unless your local pricing page says Codex is included.
ChatGPT Plus	$20/month	Good default if you want Codex and already use ChatGPT.
ChatGPT Pro $100	$100/month	For real weekly Codex projects with 5x higher limits than Plus.
ChatGPT Pro $200	$200/month	For heavy parallel Codex work and demanding long-running workflows.
Business Codex	Pay as you go	Development-focused workspace with no fixed seat fee; usage-based.

Tier	Price	What it means for this workshop
Business	About $20/user/month billed annually, often $25/user/month monthly	Team workspace with admin and privacy controls.
Enterprise/Edu	Custom	Institutional deployment with stronger controls and support.

Choose Codex if you already pay for ChatGPT. For the workshop, Plus is usually enough. and in fact gives you very generous usage limits with respect to Claude Code. Pro only makes sense if you already know you will use Codex several days per week.

Claude Code Pricing

Notice: the LSE Antrhopic Partnership does not cover the use of Claude Code. You will have to buy it with personal funds or research funds.

Official pages:

- Claude pricing: https://www.claude.com/pricing
- Choosing a Claude plan: https://support.claude.com/en/articles/11049762-choosing-a-claude-plan
- Claude Pro: https://support.claude.com/en/articles/8325606-what-is-the-pro-plan
- Claude Code desktop: https://code.claude.com/docs/en/desktop-quickstart

Tier	Price	What it means for this workshop
Free	$0/month	Useful for chat, but not the right plan if you want Claude Code.
Pro	$20/month, or about $17/month with annual billing	Good default for Claude Code and regular research use.
Max 5x	$100/month	Frequent Claude Code use across several tasks per week.
Max 20x	$200/month	Daily, heavy agent use with larger projects and fewer interruptions.
Team Standard	About $25/user/month annual or $30/user/month monthly in many regions	Shared team workspace, central billing, admin controls.

Tier	Price	What it means for this workshop
Team Premium	About $100/user/month annual or $125/user/month monthly in many regions	Higher team limits and heavier shared usage.
Enterprise	Starts around $20/seat/month annual plus usage, or custom terms	Large organizations, compliance, spend limits, audit logs.
Education	Custom	Institutional education deployments.

Choose Claude Code if you want a strong terminal workflow, good long-context reasoning, and a tool that feels natural for research prose, code review, and refactoring. Claude Pro is usually enough for the workshop. Max is for people who already know they will use Claude Code often.

Cursor Pricing

Official pages:

- Cursor pricing: https://cursor.com/pricing
- Cursor desktop install: https://docs.cursor.com/en/get-started/installation
- Cursor Agent CLI: https://docs.cursor.com/en/cli/installation

Tier	Price	What it means for this workshop
Hobby	$0/month	Good for trying Cursor, but limited Agent requests.
Pro	$20/month	Good default if you want an IDE-first workflow.
Pro+	$60/month	Cursor recommends this for daily Agent users.
Ultra	$200/month	Cursor recommends this for power users using agents heavily.
Teams	$40/user/month	Shared team context, shared rules, analytics, SSO.
Enterprise	Custom	Larger organizations needing pooled usage and admin controls.

Choose Cursor if you want the least terminal-heavy path, already like VS Code, or want autocomplete, chat, and agents inside one editor. Cursor Pro is usually enough for the workshop.

Privacy And Research Data

Do not use confidential referee manuscripts, restricted administrative data, private student data, or non-public datasets in any AI tool unless you have explicit permission and the relevant data-governance approval.

For this workshop, use synthetic data, public datasets, toy examples, or your own non-confidential code.

2. What You Need Installed

Before the workshop, install:

- Git
- A GitHub account
- Python 3.10 or later
- Node.js 20 or later
- One main agentic coding tool from Section 1
- Playwright with a Chromium browser

You do not need the course repository yet. Create a temporary setup folder using the terminal in your machine:

```
mkdir -p ~/ac4e-prework
cd ~/ac4e-prework
```

Windows PowerShell:

```
mkdir $HOME\ac4e-prework
cd $HOME\ac4e-prework
```

3. Install Git And Create A GitHub Account

Create your GitHub account at https://github.com. If you already have one, you will need your username and email associated with it to be able to use it with AI coding agents.

Install Git from https://git-scm.com/downloads.

Check Git:

```
git --version
```

Set your identity:

```
git config --global user.name "Your Name"
git config --global user.email "your.name@lse.ac.uk"
```

4. Install Python

Install Python 3.10 or later from one of these:

- Python.org: https://www.python.org/downloads/
- Miniforge: https://github.com/conda-forge/miniforge
- Anaconda (recommended for economists): https://www.anaconda.com/download

Check Python:

```
python3 --version
```

On Windows, use:

```
py -3 --version
```

Create a workshop test environment with your terminal.

macOS or Linux:

```
cd ~/ac4e-prework
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy pandas scipy statsmodels matplotlib jupyter ipykernel playwright
python -m playwright install chromium
```

Windows PowerShell:

```
cd $HOME\ac4e-prework
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install numpy pandas scipy statsmodels matplotlib jupyter ipykernel playwright
python -m playwright install chromium
```

Run a Python smoke test.

macOS or Linux:

```
python - <<'PY'
import pandas as pd
import statsmodels.api as sm

df = pd.DataFrame({"y": [1, 2, 3, 4], "x": [0, 1, 0, 1]})
model = sm.OLS(df["y"], sm.add_constant(df["x"])).fit()
print("Python economics stack works.")
print(model.params.to_dict())
PY
```

Windows PowerShell:

```
@'
import pandas as pd
import statsmodels.api as sm

df = pd.DataFrame({"y": [1, 2, 3, 4], "x": [0, 1, 0, 1]})
model = sm.OLS(df["y"], sm.add_constant(df["x"])).fit()
print("Python economics stack works.")
print(model.params.to_dict())
'@ | python
```

You should see `Python economics stack works.`

5. Install Node.js

Install Node.js 20 or later from https://nodejs.org/.

Check Node and npm:

```
node --version
npm --version
```

Install the Playwright command-line tools in your temporary folder:

```
cd ~/ac4e-prework
npm init -y
npm install --save-dev @playwright/test
npx playwright install chromium
npx playwright --version
```

Windows PowerShell:

```
cd $HOME\ac4e-prework
npm init -y
npm install --save-dev @playwright/test
npx playwright install chromium
npx playwright --version
```

6. Test Playwright

With your Python environment activated, run this test.

macOS or Linux:

```
python - <<'PY'
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch()
    page = browser.new_page()
    page.set_content("""
        <html>
          <body>
            <label>Name <input id="name"></label>
            <button id="submit">Submit</button>
            <p id="result"></p>
            <script>
              document.querySelector("#submit").onclick = () => {
                document.querySelector("#result").textContent =
                  "Hello " + document.querySelector("#name").value;
              };
```

```
            </script>
          </body>
        </html>
    """)
    page.fill("#name", "LSE")
    page.click("#submit")
    print(page.text_content("#result"))
    browser.close()
PY
```

Windows PowerShell:

```
@'
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch()
    page = browser.new_page()
    page.set_content("""
        <html>
          <body>
            <label>Name <input id="name"></label>
            <button id="submit">Submit</button>
            <p id="result"></p>
            <script>
              document.querySelector("#submit").onclick = () => {
                document.querySelector("#result").textContent =
                  "Hello " + document.querySelector("#name").value;
              };
            </script>
          </body>
        </html>
    """)
    page.fill("#name", "LSE")
    page.click("#submit")
    print(page.text_content("#result"))
    browser.close()
'@ | python
```

Expected output:

```
Hello LSE
```

Optional: try Playwright's recorder:

```
npx playwright codegen https://example.com
```

Close the browser window when you are done.

7. Install Your Chosen Agentic Coding Tool

Install only the tool you chose in Section 1 unless you already use another one. For your chosen tool, install both the CLI and the desktop/editor app where available.

Option A: Codex

Install the CLI:

```
npm i -g @openai/codex
codex --version
```

Start Codex in the temporary folder:

```
cd ~/ac4e-prework
codex
```

Windows PowerShell:

```
cd $HOME\ac4e-prework
codex
```

Sign in when prompted. Then ask:

```
Inspect this folder and tell me what setup files you see. Do not edit files.
```

Install the desktop app:

1. Open https://openai.com/academy/codex-how-to-start/.
2. Download the official Codex desktop app for your platform if available.
3. Sign in with the same ChatGPT account.
4. Create a project pointing to `~/ac4e-prework` or `$HOME\ac4e-prework`.
5. Create a test thread and ask the same inspection prompt.

Codex desktop is available on macOS and Windows. Linux users should use the CLI for the hands-on exercises.

Option B: Claude Code

Install the CLI.

macOS, Linux, or WSL:

```
curl -fsSL https://claude.ai/install.sh | bash
claude --version
```

Windows PowerShell:

```
irm https://claude.ai/install.ps1 | iex
claude --version
```

Alternative install paths from the official guide include:

```
brew install --cask claude-code
npm install -g @anthropic-ai/claude-code
```

Do not use `sudo` with the npm install path.

Start Claude Code in the temporary folder:

```
cd ~/ac4e-prework
claude
```

Sign in when prompted. Then ask:

```
Inspect this folder and tell me what setup files you see. Do not edit files.
```

Install the desktop app:

1. Open https://code.claude.com/docs/en/desktop-quickstart.
2. Download the Claude desktop app for macOS or Windows.
3. Sign in with your Anthropic account.
4. Open the `Code` tab.
5. Select `Local` and choose `~/ac4e-prework` or `$HOME\ac4e-prework`.
6. Keep the default permission mode so you can review proposed changes.

Claude Code desktop is available on macOS and Windows. Linux users should use the CLI.

Option C: Cursor

Install the desktop/editor app:

1. Open https://cursor.com.
2. Click `Download`.
3. Run the installer.
4. Open Cursor and sign in.
5. Open `~/ac4e-prework` or `$HOME\ac4e-prework`.
6. Wait for indexing to finish.

Install the `cursor` shell command from inside Cursor:

1. Open the Command Palette with `Cmd+Shift+P` on macOS or `Ctrl+Shift+P` on Windows/Linux.
2. Run `Shell Command: Install 'cursor' command in PATH`.
3. Open a new terminal and check:

```
cursor --version
cursor .
```

Install Cursor Agent CLI if you want a terminal-native agent:

```
curl https://cursor.com/install -fsS | bash
cursor-agent --version
cursor-agent
```

If `cursor-agent` is not found, add `~/.local/bin` to your PATH.

zsh:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

bash:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

In Cursor Agent, ask:

```
Inspect this folder and tell me what setup files you see. Do not edit files.
```

8. Run A Self-Check

Run this in your temporary folder. It does not require the course repository.

macOS or Linux:

```
cd ~/ac4e-prework
source .venv/bin/activate
python - <<'PY'
import importlib.util
import shutil
import subprocess
import sys

commands = [
    ("git", ["git", "--version"]),
    ("node", ["node", "--version"]),
    ("npm", ["npm", "--version"]),
    ("codex", ["codex", "--version"]),
    ("claude", ["claude", "--version"]),
    ("cursor", ["cursor", "--version"]),
    ("cursor-agent", ["cursor-agent", "--version"]),
]

packages = ["pandas", "statsmodels", "playwright"]

print("Python:", sys.version.split()[0])

for label, command in commands:
    if shutil.which(command[0]) is None:
        print(f"OPTIONAL/MISSING: {label}")
        continue
    result = subprocess.run(command, text=True, capture_output=True)
    output = (result.stdout or result.stderr).strip().splitlines()
```

```
    print(f"OK: {label} - {output[0] if output else 'found'}")

for package in packages:
    if importlib.util.find_spec(package) is None:
        print(f"MISSING: Python package {package}")
    else:
        print(f"OK: Python package {package}")
PY
```

Windows PowerShell:

```
cd $HOME\ac4e-prework
.\.venv\Scripts\Activate.ps1
@'
import importlib.util
import shutil
import subprocess
import sys

commands = [
    ("git", ["git", "--version"]),
    ("node", ["node", "--version"]),
    ("npm", ["npm", "--version"]),
    ("codex", ["codex", "--version"]),
    ("claude", ["claude", "--version"]),
    ("cursor", ["cursor", "--version"]),
    ("cursor-agent", ["cursor-agent", "--version"]),
]

packages = ["pandas", "statsmodels", "playwright"]

print("Python:", sys.version.split()[0])

for label, command in commands:
    if shutil.which(command[0]) is None:
        print(f"OPTIONAL/MISSING: {label}")
        continue
    result = subprocess.run(command, text=True, capture_output=True)
    output = (result.stdout or result.stderr).strip().splitlines()
    print(f"OK: {label} - {output[0] if output else 'found'}")

for package in packages:
    if importlib.util.find_spec(package) is None:
        print(f"MISSING: Python package {package}")
    else:
        print(f"OK: Python package {package}")
'@ | python
```

It is fine if tools you did not choose are listed as `OPTIONAL/MISSING`. For example, a Cursor user does not need `codex`, and a Codex user does not need `cursor-agent`.

Before the workshop, the important checks are:

- Git works.
- Python imports `pandas`, `statsmodels`, and `playwright`.
- Node and npm work.
- Your chosen agentic coding tool opens in `~/ac4e-prework`.
- You can sign in to your chosen tool.

9. If Something Breaks

First, do not spend hours debugging alone. Setup problems are common.

Send a message in the MS Teams chat dedicated to this workshop. Include:

- Your operating system.
- Which tool you chose: Codex, Claude Code, or Cursor.
- The command that failed.
- The exact error message.
- A screenshot if the problem is in a desktop app.

Useful quick fixes:

- Restart the terminal after installing a CLI.
- Open a new terminal if a command is not found.
- Check that your Python virtual environment is activated.
- On Windows, try PowerShell as Administrator only if an installer explicitly needs it.
- If Node or Python was just installed, restart the computer.
- If Playwright says a browser is missing, rerun `python -m playwright install chromium` and `npx playwright install chromium`.

10. Final Checklist

Complete this before the workshop:

- I chose one main agentic coding tool.
- I understand the price of the plan I chose.
- Git works.
- I have a GitHub account.
- Python 3.10 or later works.
- `pandas`, `statsmodels`, and `playwright` import in Python.
- Node.js and npm work.
- Playwright can launch Chromium.
- My chosen agentic coding tool opens and I can sign in.
- I know to contact the workshop MS Teams chat if setup fails.

Bring the same laptop to the workshop.